```
      : ExactlyOnceProjection[Offset, EventEnvelope[Bet.Event]] = {

    val tag = Bet.tags(index)

    val sourceProvider =                                              ❶
      EventSourcedProvider.eventsByTag[Bet.Event](
        system = system,
        readJournalPluginId = JdbcReadJournal.Identifier,
        tag = tag)

    JdbcProjection.exactlyOnce(
      projectionId = ProjectionId("BetProjection", tag),
      sourceProvider = sourceProvider,                                ❶
      handler = () => new BetProjectionHandler(repository),           ❷
      sessionFactory = () => new ScalikeJdbcSession())(system)        ❸
  }
}
```

❶ Source of events from the journal

❷ Handles events, filters them, and upserts to the database

❸ Database session used to write

All these components are combined in a `JdbcProjection` with effectively-once semantics.

The handler takes each event and passes the `Opened` ones to the repository to save them. It is worth noting that you pass only the repository to the handler in its constructor. The `ScalikeJdbcSession` is not passed directly to this method; it must be included in the scope, and that is done by creating a connection pool in `Main`. So when you initialize the projection, the database connection pool is already available. You see this later in section 15.1.3.

To expose the HTTP endpoint so operators can consume it, you can wrap the `BetRepositoryImpl` in a gRPC. To do so, you need the following: